DyME

Dynamic Mutagenesis Engine

User Manual

Pisabarro Group
https://github.com/pisabarro-group/DYME

1. Introduction

DyME (Dynamic Mutagenesis Engine) is a distributed computational platform designed to automate large-scale molecular dynamics (MD) analysis and high-throughput mutational exploration of protein complexes. It was developed by the Pisabarro Group to address a long-standing bottleneck in structural bioinformatics: while MD-based tools exist for studying individual mutations, systematic engineering of protein recognition across hundreds or thousands of variants requires an integrated, automated solution that was previously unavailable.

The platform takes 3D structural information from any protein–protein or protein–DNA complex as input, and orchestrates the full pipeline from model preparation and mutagenesis through MD simulation and post-processing, all the way to interactive comparative analysis. DyME supports single, double, and triple modifications, allowing users to assemble combinatorial mutant libraries and explore the energetic and conformational consequences of site-specific alterations at scale.

Architecturally, DyME follows a distributed client–worker model. A central **Main Node** (deployed as a Docker container) hosts the web UI, REST API, and a MongoDB database. One or more **Worker Nodes** (deployed as Apptainer/Singularity containers) execute asynchronous MD simulations (MD nodes, GPU-accelerated) and trajectory post-processing (Scavenger nodes, CPU-only). All nodes communicate through the shared database, and all simulation data is stored on a common filesystem accessible to every node in the network.

The web-based graphical interface provides an interactive project wizard for defining mutation libraries and MD parameters, real-time job monitoring dashboards, and a Trajectory Comparative Analyzer (TCA) for multi-feature exploration of results including binding free energies, intermolecular contacts, water bridging, and per-residue decomposition enabling straightforward comparison across large sets of simulations from a single browser session.

2. Installation

2.1 Prerequisites

Before running the installer, ensure the following are available on the machine designated as the Main Node:

- **Operating System:** Linux x86_64 (Ubuntu ≥ 22.04 or RHEL-based recommended)
- **Docker** (≥ 27) with BuildX plugin
- **Apptainer / Singularity** (≥ 3.10) — required for worker nodes
- **MODELLER license key** — free academic license available at https://salilab.org/modeller/registration.html
- **Disk space:** ≥ 20 GB on the Main Node for containers; shared partition (NFS/BeeGFS/Lustre) for MD data (GB to TB scale)
- **Network:** all nodes (main + workers) must be able to reach the Main Node on ports 8080 and 27017

The installer will attempt to install missing dependencies (Docker, BuildX, Apptainer, curl) automatically. If automatic installation fails, resolve dependencies manually and re-run the installer.

2.2 Step-by-Step Installation

Log in to the server designated as the Main Node. Navigate to a directory with at least 20 GB of free space and run:

```
git clone https://github.com/pisabarro-group/DYME.git
cd DYME
chmod a+x install.sh
./install.sh
```

The installer will guide you through an interactive setup. When prompted, provide:

1. The path to the **shared filesystem** that all nodes will access (dyme_root). This path must be reachable from every worker node.
2. Your **MODELLER license key**.
3. Whether to install **test data** (~11 GB; two pre-built projects from Zenodo).

Upon completion, the installer will have produced:

- **dyme_main** — a Docker image for the Main Node
- **dyme_node.sif** — an Apptainer/Singularity image for worker nodes

2.3 Starting and Stopping the Main Node

Start the Main Node by binding the shared filesystem and exposing the required ports:

```
docker run \
    --name dyme_main \
    -d \
    -v "/path/to/shared/filesystem:/dyme_root" \
    -p 8080:80 \
    -p 27017:27017 \
    dyme_main:latest
```

Access the web UI at `http://<main_node_hostname>:8080` in any browser on your network.

To stop the Main Node:

```
docker stop dyme_main
```

NOTE	The Main Node must remain running for worker nodes to pick up tasks, update their status, or for users to access the web UI.

3. Using DyME — Step by Step

3.1 Preparing a Molecular Complex for DyME

DyME requires a **cleaned, complete PDB file** of your protein–protein or protein–DNA complex as the starting point. Before loading it into DyME, ensure the following:

- **Remove water molecules and co-crystallization artifacts** unless they are functionally relevant.
- **Resolve missing residues and alternate conformations**. DyME uses MODELLER internally to build missing segments, but providing the most complete possible structure reduces ambiguity.
- **Verify chain identifiers**. Each molecule in the complex (receptor chain(s) and ligand chain(s)) must have distinct, consistent chain IDs.
- **Ensure consistent residue numbering**. Non-standard numbering (insertions, negative numbers) can interfere with the mutation specification in subsequent steps.
- **Protonation state**: DyME handles protonation during system preparation (via tleap), but verifying histidine states and disulfide bridges beforehand is recommended.

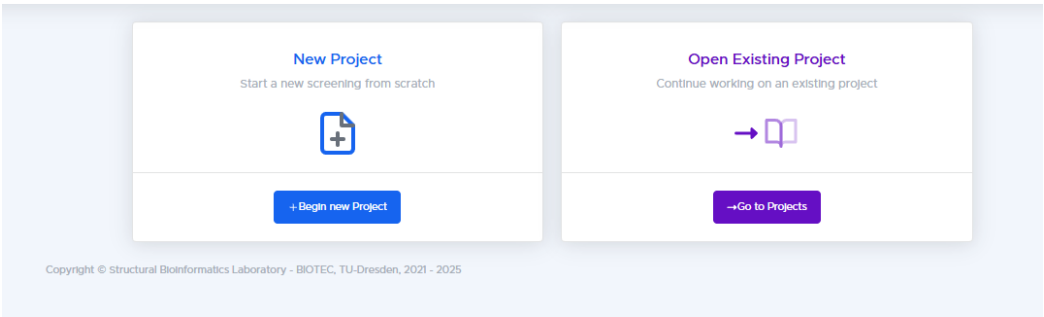
Keep in mind:

1) All molecules in your complex must be in **the same PDB file**.
2) Each molecular object should have a unique **chain identifier**.
3) Ensure your molecules are delimited by the **TER** notation.
4) Ensure special HETATMs are defined as **ATOM** (i.e. Zinc)

IMPORTANT	Your PDB **must be prepared** with the tool "**pdb4amber**", as you would prepare a PDB file to run with AmberMD.

Place the prepared PDB in an accessible location. You will upload or reference it during project creation.

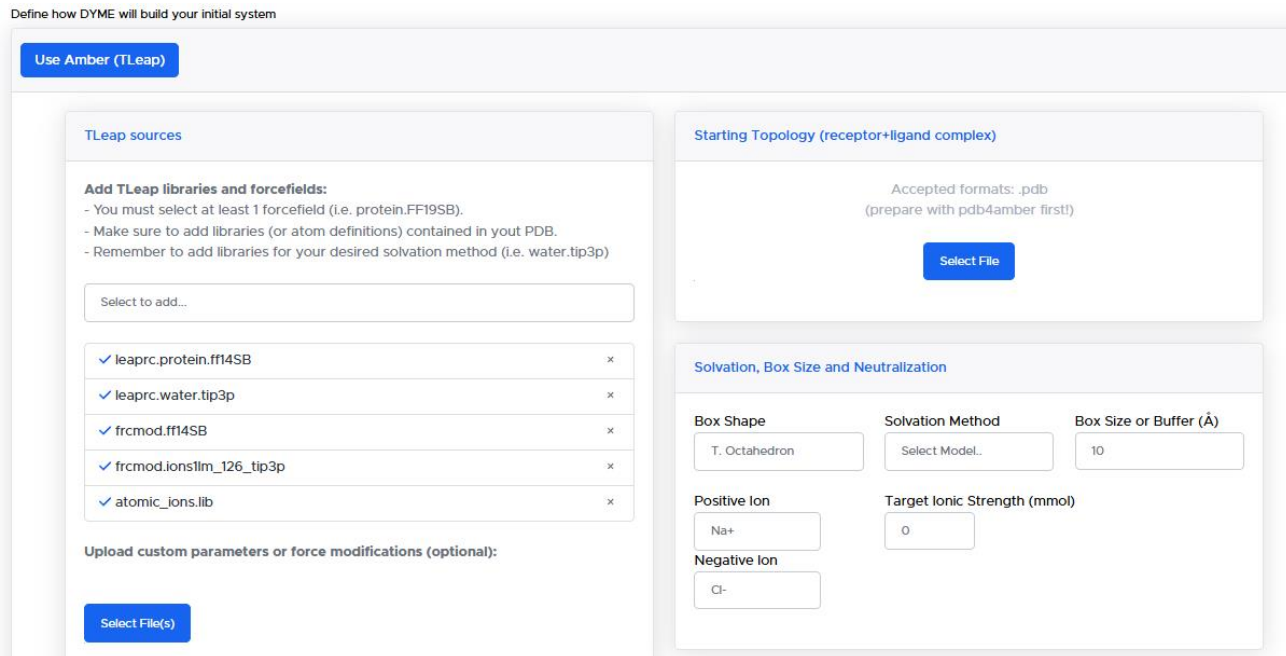
3.2 Creating a Project with the Wizard



Open the DyME web UI at `http://<main_node_hostname>:8080`. On the home dashboard, click **+Begin new Project** to launch the project creation wizard. The wizard walks you through three configuration steps (Step 1, Step 2, Step 3) described below.

Give the project a descriptive name (more than 6 characters long).

3.3 Step 1 — Initial Preparation, Definition and Setup



In Step 1 you upload the prepared PDB file, add suitable forcefields, libraries and force modification files. Likewise, you can define custom Amber AtomTypes or custom bond definitions, as in you were using tLeap to generate AmberMD input files.

1. **Upload the PDB file** of your complex using the “Select file” selector.

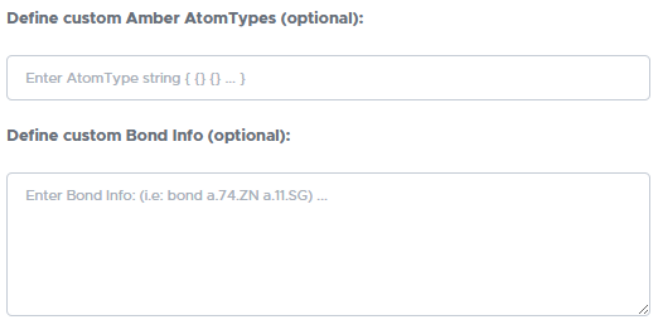
2. **Add TLeap Sources.** The dropdown contains all force fields, libraries and force modifications available in AmberTools. Open the dropdown and select as many definitions as desired, one by one. The system will instruct TLeap to load the selected libraries when building parameters and topology files.

   Selected libraries will accumulate below the dropdown. To remove a library added by accident, use the [x] button at the right of each added library.

   In the example above, the FF14SB forcefield and TIP3P water model were added, as well as the atomic_ions library.

   Custom force modifications (in the form of .off or .frcmod files) can be added by using the “Select file(s)” button and uploading the corresponding files to the project (i.e. the ZAFF source files for the Zinc Amber Forcefield).

3. **Custom Atoms and Custom bond definitions (optional).**



If your system does not contain custom atoms or bonds, you can safely leave these fields blank. If your system contains custom atom definitions or custom bond information, they must be added following the **TLeap notations** in the corresponding fields.

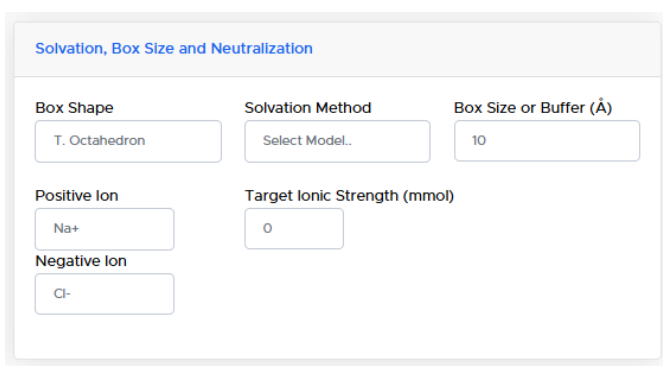
For example, if using the ZAFF forcefield, Zn atoms must be declared explicitly in TLeap notation.

{{ "ZN" "Zn" "sp3" } { "S4" "S" "sp3" } { "N3" "N" "sp3" }}

As well as the custom bonds that Zinc forms with sidechain atoms in the complex.

bond a.87.ZN a.8.SG
bond a.87.ZN a.11.SG
bond a.87.ZN a.24.NE2
bond a.87.ZN a.28.NE2

4. **Assign a box shape, solvation method, box buffer, and neutralization options**.

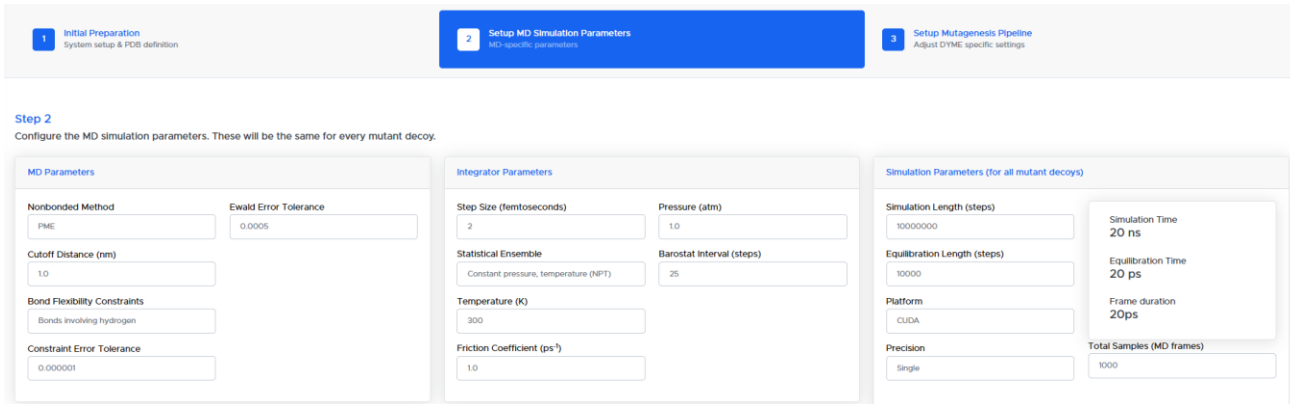


Select the desired options from the dropdowns and set the values as desired. For the box size, the distance to the edge of the box (buffer size) is in Angstroms.

TIP	Remember the “solvation method” must match the solvent libraries selected earlier.

3.4 Step 2 — MD Simulation parameters

Step 2 defines the Molecular dynamics protocol. These settings will be passed downstream to OpenMM, at the core of the MD simulation engine. All mutants generated for a project will undergo the same MD settings defined at this stage: including the number of frames, step size, pressure, ensemble, constraints, and other integration settings.



By default, the system will suggest using PME with standard settings using step size of 2 femtoseconds, as well as the NPT ensemble (constant pressure and temperature).

The simulation length can be adjusted by adding steps to the “simulation length” (i.e 10 million steps of 2fs is equal to 20ns of simulation time). The values will be updated automatically upon changing the number of steps, equilibration steps, or step size. Adjust according to your scientific objectives.

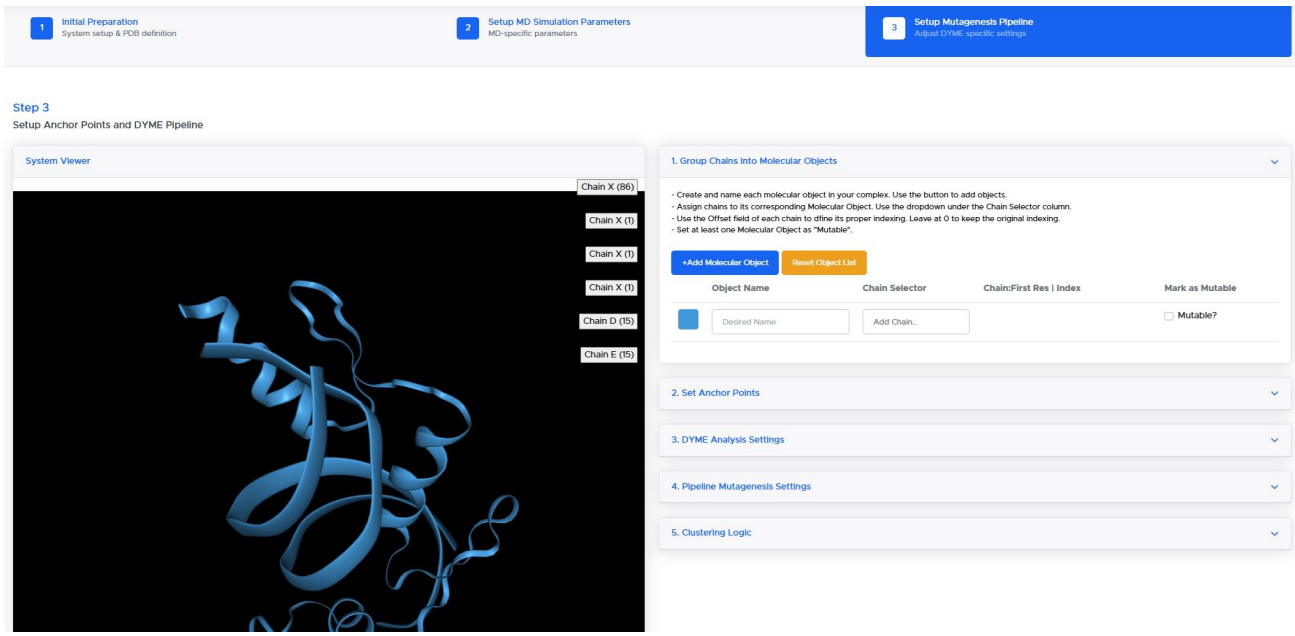
The “total samples” value determines how often the data reporters will produce a simulation frame. Be advised that selecting 10000 frames per simulation will likely produce extremely big MD trajectory files, which may affect scavenging times and disk consumption.

TIPS	* It is recommended that the cutoff distance of non-bonded interactions is set to less than the buffer distance to the edge of the simulation box (set in step 1). The system will try to auto-tune this parameter accordingly, but the cutoff can be set to any desired value. * Note: by default, DyME uses the “Langevin Integrator”, as provided by OpenMM.

3.5 Step 3 — Mutagenesis Pipeline

Step 3 configures all aspects of the mutagenesis pipeline. This step of the wizard is divided into 5 individual sections.

IMPORTANT	Options must be filled carefully and thoroughly. Pay special attention to the values of every field and how they alter the workflow.



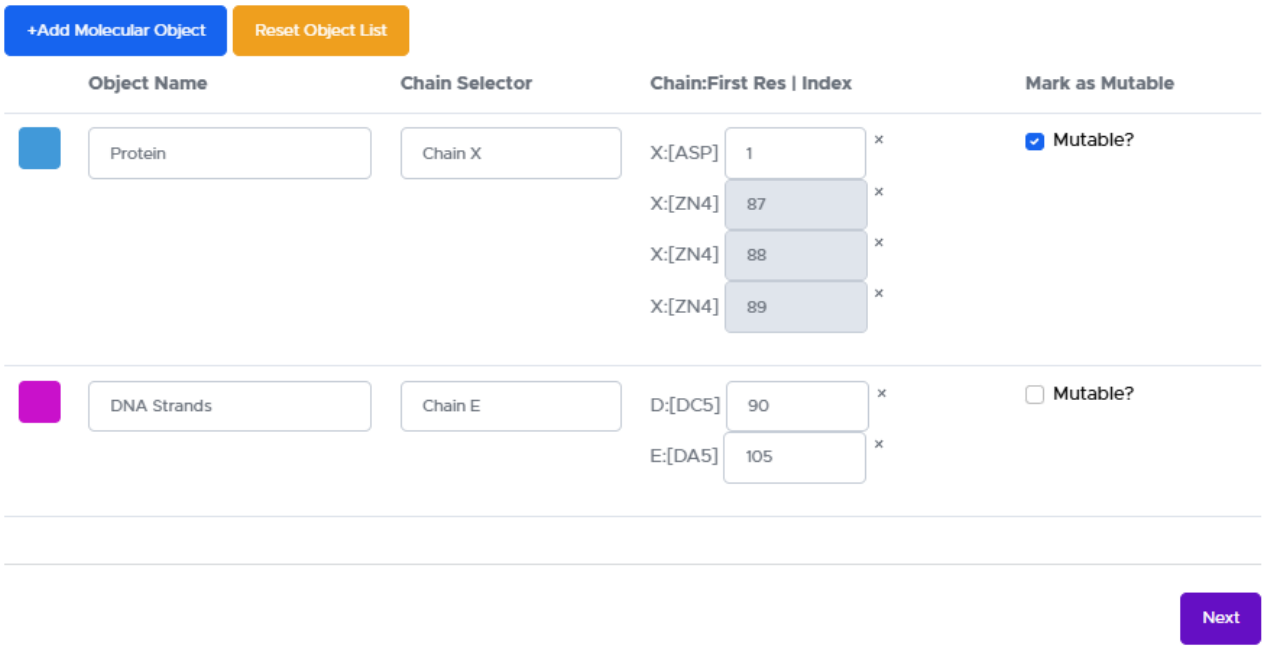
The uploaded PDB file will be displayed (at the left) and interactively showcase several aspects of the wizard options during Step 3. Initially, all chain identifiers and molecules will be shown in the same color (blue).

Note that each chain has a label at the right of the 3D display.

The corresponding molecule will be highlighted if the mouse is hovered over its label. Hover the mouse on all labels to confirm which chain is which molecule in the 3D structure, and proceed to define the molecular objects using the first section at the right side of the screen.

1. Group Chains into Molecular Objects

A “Molecular Object” is a chain or group of chains that DyME will treat logically as an individual molecule. You must define **at least two** molecular objects. One such object will be treated as **mutable**.



Add as many molecular objects as desired using the **+ Add Molecular Object** button, and provide a name for each object (i.e. Protein, and DNA Strands).

Select from the dropdowns which chain identifiers will belong to each object. For this case example, Chains **D and E** have been added to the object “DNA Strands”, and Chain **X** has been added to the object named “Protein”. To add a chain to an object, simply select it from the dropdown.

When ready, mark **one such object** as mutable by using the checkbox at the last column. In this case, it will be the object named Protein. DyME can handle more than one mutable object, however, on most studies of these sort, only one side of the complex is to be mutated.

You can also select the color of the object by clicking the square at the left of the name. A color palette will be provided and colors will be applied automatically.

If you would like DyME to match residue numbering with the original PDB file (instead of AMBER renumbering from index 1), you can force the internal residue map of DyME to use a different first position by replacing the corresponding number in the 4th column.

For example, if you’d like the first residue of chain X (ASP) to start in position 780 instead of 1, this number should replace the 1. The same logic applies to other chains in all molecular objects. If you’d like to keep the altered numbering calculated by pdb4amber, you can safely ignore these values.

If you wish to start over. Click the “**Reset Object List**” button.

When satisfied, click **Next** to finish object definition.

2. Set Anchor Points

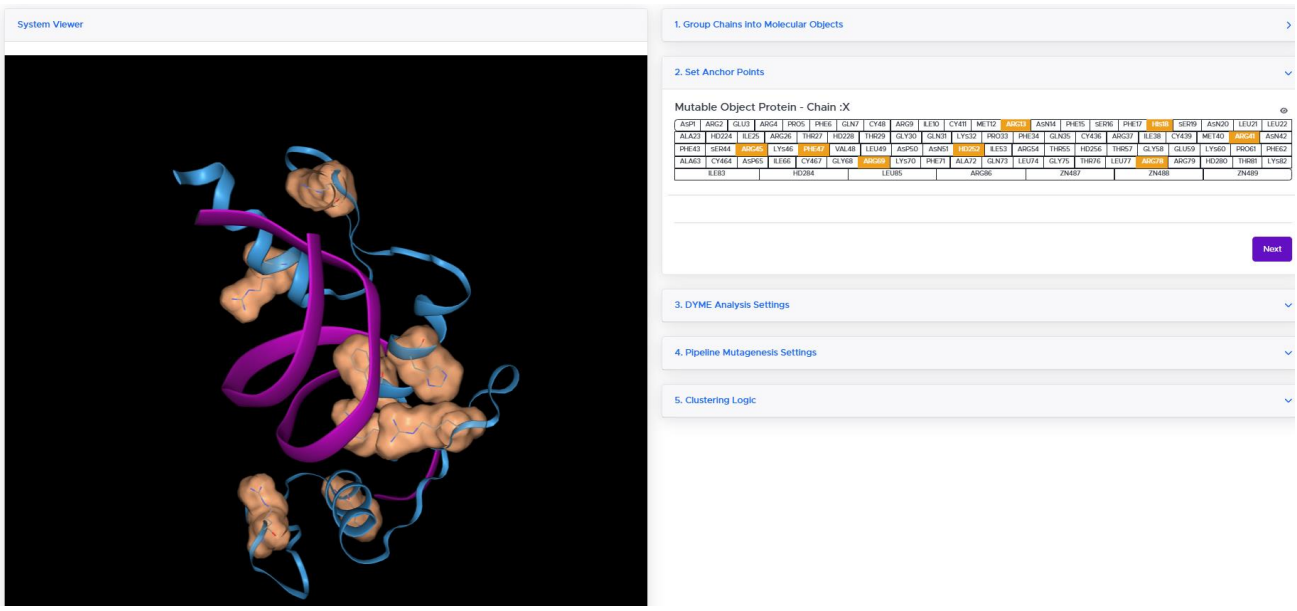
Anchor points are interfacial residues on the **MUTABLE objects** where you would like to focus your attention, even if they are ultimately not selected for mutagenesis on a later stage.

Scavenging analyses are computed primarily on anchor points, which is useful for straightforward comparative analysis between MD simulations.

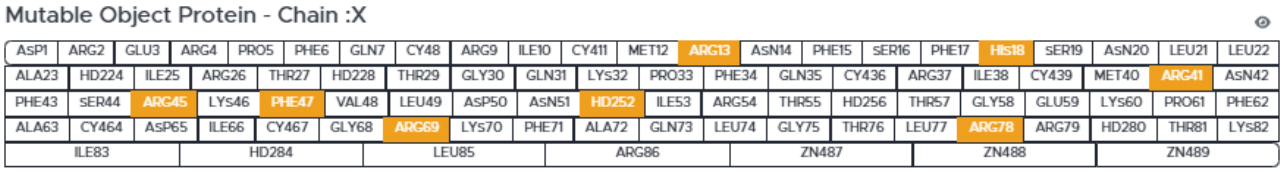
You can select as many anchor points as desired.

DyME will generate a table with all positions of the “mutable object” and suggest residues that could be interesting anchor points (by proximity to the non-mutable object). These positions will be highlighted in **ORANGE**, and their surface displayed in the 3D structure accordingly.

In this example, DyME has suggested 8 anchor points.



Hovering the mouse on any position on the table (right) will highlight the surface area of that position in the 3D structure (left) for ease of visualization and confirmation.

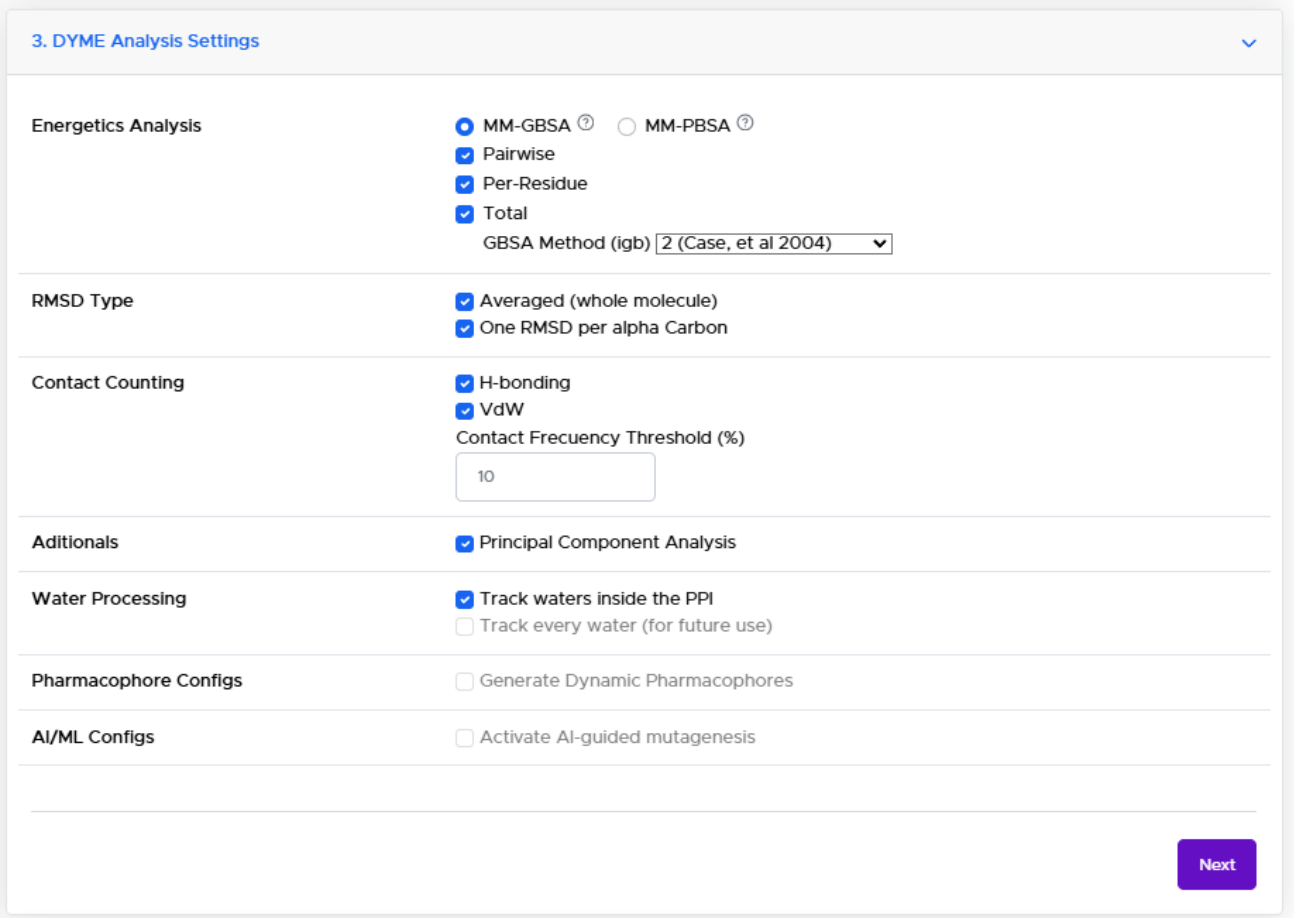


To set a position as anchor point, simply activate it in the table by clicking on it. To remove a position as anchor point, click again to deactivate it.

Once all desired anchor points have been defined. Click **Next**

3. DyME Analysis Settings

This section provides options to adjust some of the parameters of the scavenging routines of DyME, such as specific configurations for the MMPBSA module, among others.



You can decide whether the scavenger will perform GBSA or PBSA methods, and whether total, pairwise and per-residue energy decompositions will be computed. Likewise, the “igb” option of GBSA can be set to different values, depending on their suitability and user criteria.

Some options are disabled and will be implemented in future versions of DyME.

Once all desired options have been defined. Click **Next**

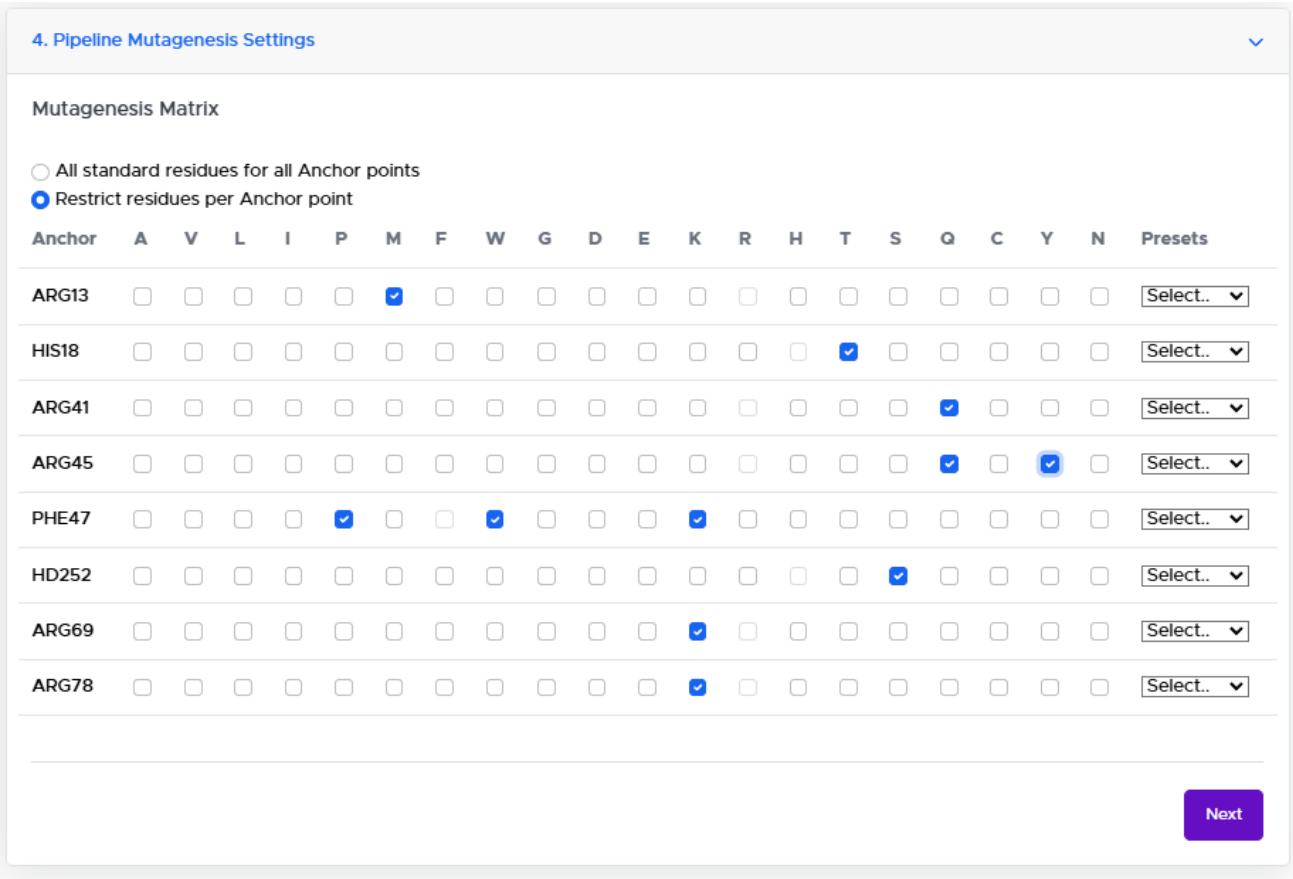
4. Pipeline Mutagenesis Settings

This section allows you to define the mutagenesis matrix for all selected anchor points. For every anchor point position you wish to mutate, you can select all possible amino acids that such position will be able to mutate into.

If you would like to use all possible substitutions in all anchor points, leave the option "**All standard residues for all Anchor points**" selected. If you'd like to select which amino acids can be substituted at each position in detail, select "**Restrict residues per Anchor point**".

When activating custom restrictions, a table is built for you to select which residues are allowed at each position.

At the end of each row, a dropdown called "**Presets**" allows you to quickly activate large groups of amino acids by category (Aliphatic, Charged, Polar, Aromatic). Selecting such option from the dropdown activates the corresponding amino acid substitutions for that anchor point.



If you would like to exclude an anchor point from mutating, leave its row blank, without any possible substitutions.

Once all desired substitution options have been defined. Click **Next**

5. Clustering Logic

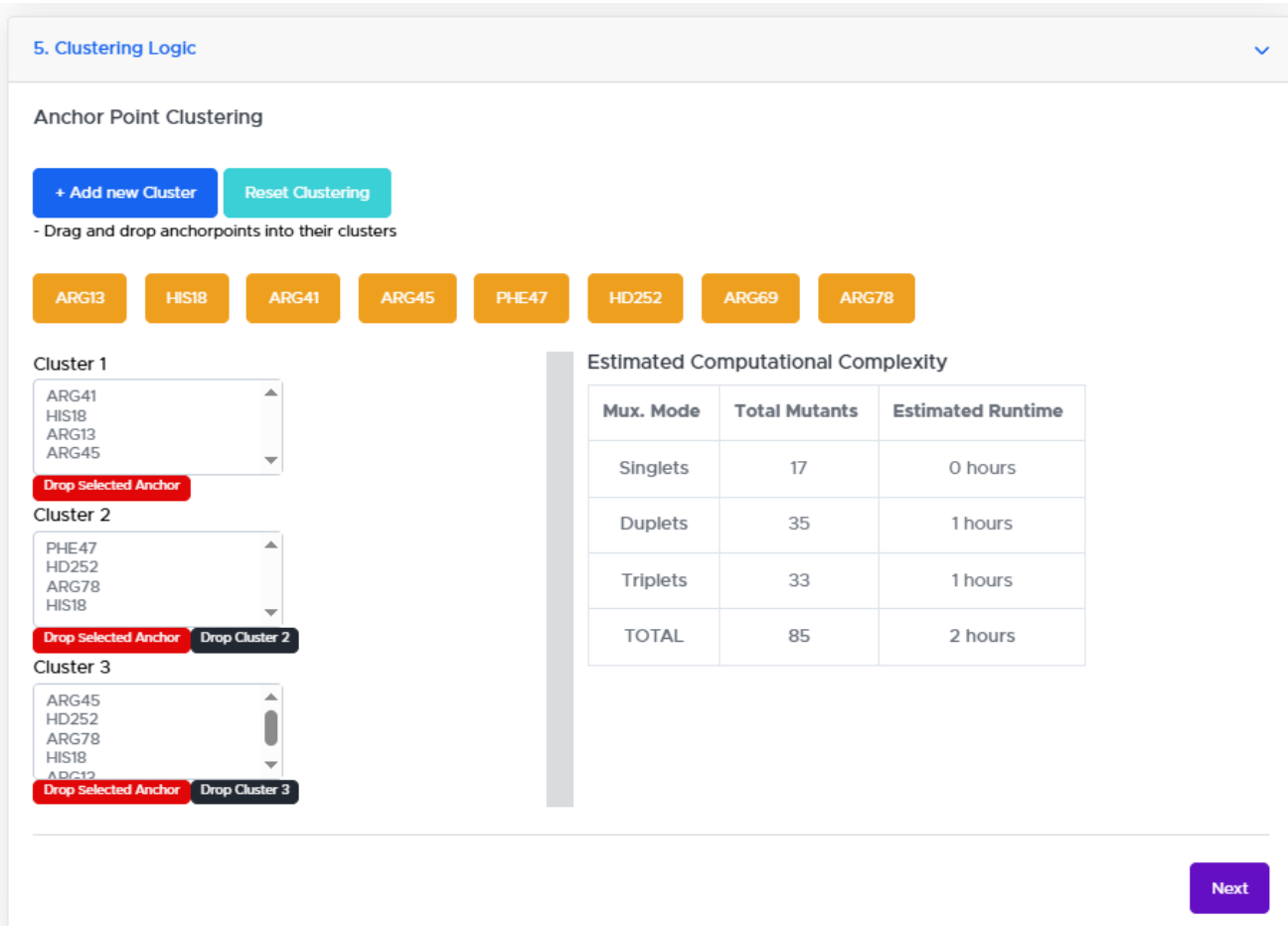
The last section of Step 3 is specifically designed to build clusters. It allows groups of anchor points to be permutated in the same combinatorial group.

To add clusters, use the "**+Add new cluster**" button. You can add as many clusters as desired to achieve your combinatorial logic, and combine as many anchor points as needed in a single cluster.

Anchor points are displayed as orange boxes. These boxes can be dragged and dropped to any cluster as needed, in order to add it to a combinatorial group.

You can remove anchor points from a given cluster by selecting the anchor point, and using the button "**Drop Selected Anchor**".

To reset the clustering, use the button "**Reset Clustering**". It will wipe the changes and start over from scratch.

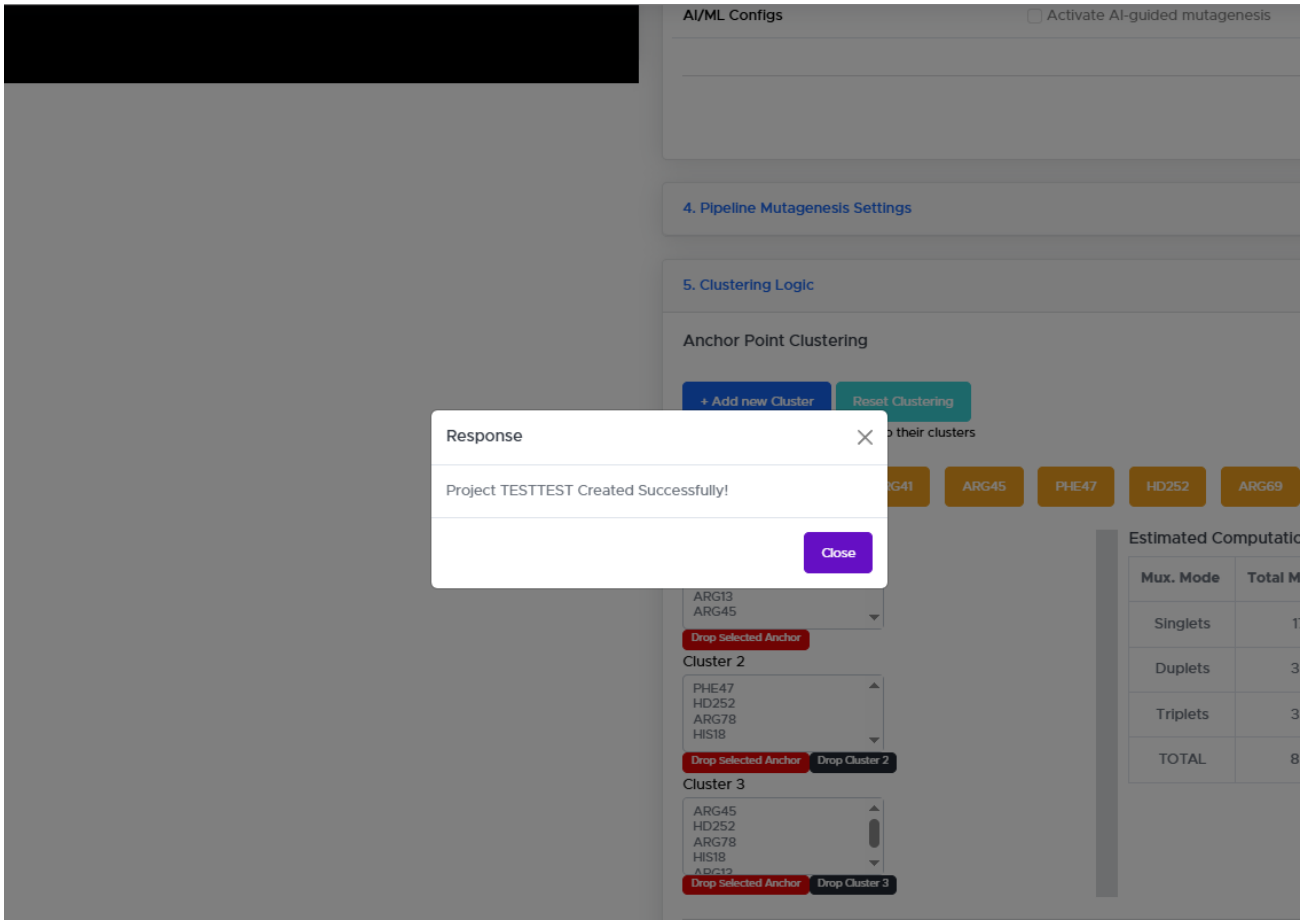


The system automatically updates the table to the right and estimates how many mutants will be created in the mutational library by computing all singlets, doublets and triplet permutations for every cluster. Likewise, the platform will discard any redundant mutants (if any) automatically.

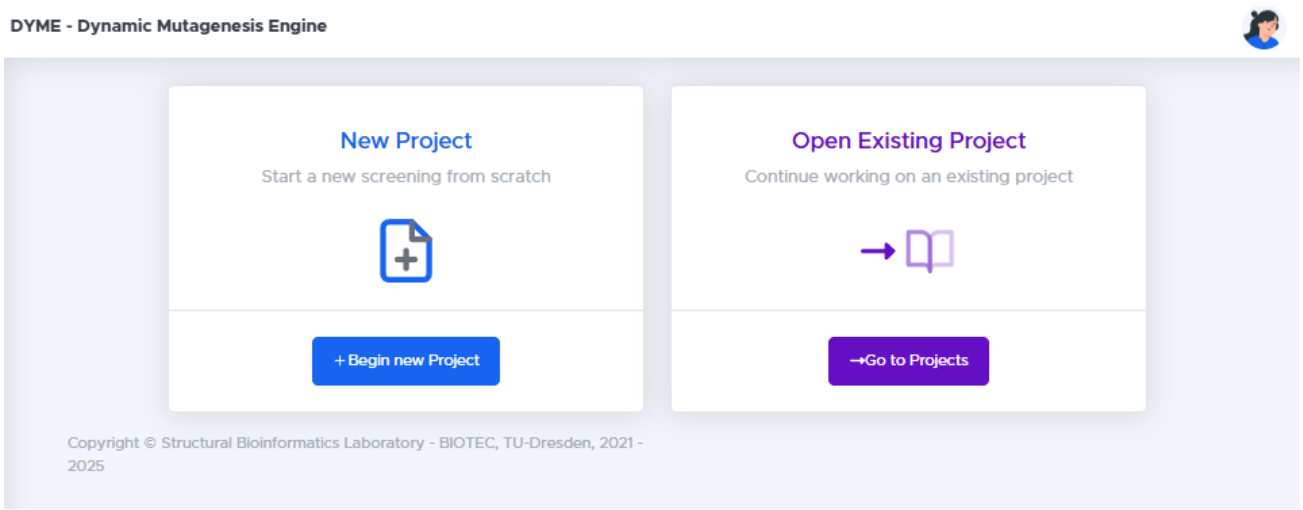
If the number of available GPU cards available has been set, it will provide an estimated runtime in hours. This setting is experimental and may not be accurate.

Once all desired clustering options have been defined. Click **Next**.

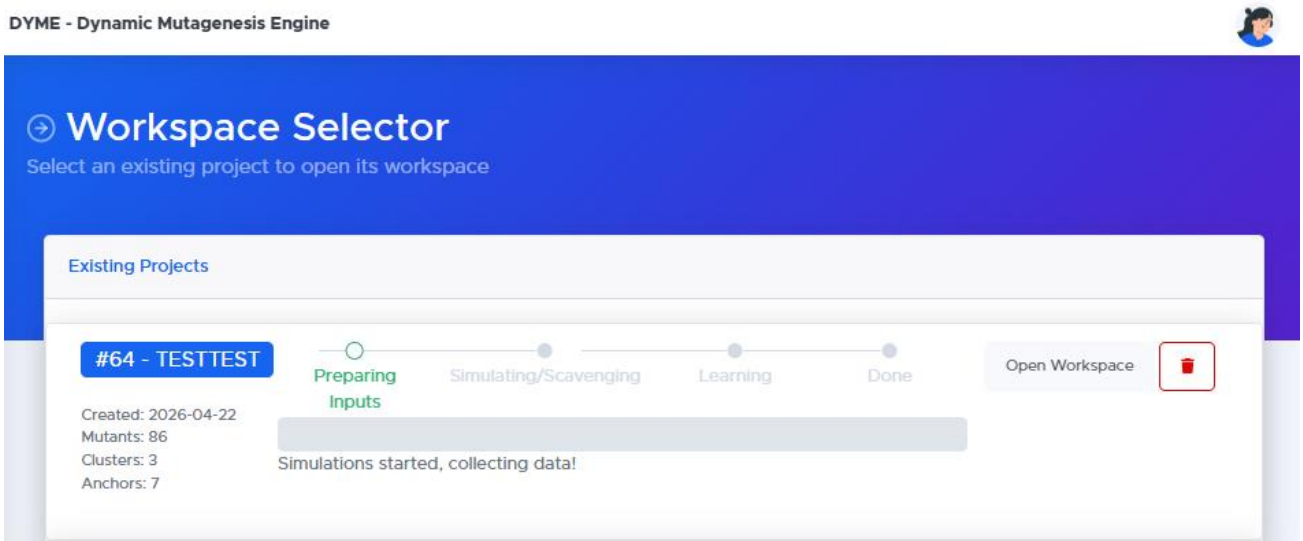
If everything has been defined properly. The system should create your project.



To return to the main screen and explore your project, click the DYME Logo, on the top left corner of the screen.



To view your new project and its status, select “Go to Projects”. The system will display a list of all projects in the system.



A green bar will indicate the current status of the workflow being performed. This progress will depend on the number of MD nodes and Scavenger nodes available for processing the workflow and the number of mutants in the project.

TIP	Once your project has been created, start the workflow by deploying MD nodes and Scavenger nodes in your computing servers!

3.6 Launching an MD Node

MD nodes consume GPU resources and execute simulations. Launch one per server that has compatible CUDA GPU cards. Copy `dyme_node.sif` image and `launch_node.sh` to the GPU server (or reference them from a shared location), then run:

```
./launch_node.sh MD <main_node_hostname> /path/to/shared/filesystem
```

Alternatively, launch directly via Apptainer:

```
export BINDPATH=/path/to/shared/filesystem
export SIF_IMAGE=dyme_node.sif
export NODETYPE=MD
export DBHOST=<main_node_hostname>

apptainer --nv exec \
    --bind "$BINDPATH":"/dyme_root" \
    --bind /dev/shm:/dev/shm \
    "$SIF_IMAGE" \
    "$NODETYPE" "$DBHOST"
```

The MD node will poll the database continuously, claim `pending` mutants atomically, and launch one MD simulation per idle GPU card. You can run multiple MD nodes on different servers simultaneously. The atomic claiming mechanism prevents double processing.

NOTE	MD nodes require CUDA-compatible GPU cards. The --nv flag passes GPU access into the Apptainer container. Ensure CUDA drivers > 10.2 are installed on the host.

3.7 Launching a Scavenger Node

Scavenger nodes process completed MD trajectories and extract data into the database (MMPBSA energies, intermolecular contacts, water bridging sites, RMSD). They are CPU-only and can run on any server with access to the shared filesystem:

```
./launch_node.sh scavenger <main_node_hostname> /path/to/shared/filesystem
```

The scavenger automatically detects mutants with status `ready_to_scavenge` and processes them in parallel batches (batch size = available CPUs ÷ 8). Like MD nodes, multiple scavenger nodes can run concurrently without conflict.

You can run both node types on the same server by launching them as separate processes or using your own SLURM jobs. Redirect output to a log file for diagnostics:

```
./launch_node.sh scavenger <main_node_hostname> /path/to/shared/data >
scavenger.log 2>&1 &
```

TIP	For HPC environments, wrap the launch commands in SLURM batch scripts tailored to your cluster's resource manager. DyME ships no SLURM templates — these must be authored by the user for their specific environment.

3.8 Monitoring Jobs

Open the DyME web UI and navigate to your project dashboard. Each mutant is tracked through the following status lifecycle:

- `pending` — queued and awaiting an MD node
- `processing` — actively being simulated on a GPU node
- `ready_to_scavenge` — simulation complete; awaiting a scavenger node
- `scavenging` — trajectory data being extracted
- `done` — all data available in the database
- `failed` — simulation or scavenging failed (inspect worker logs for details)

The DyME dashboard displays mutant counts per status. But for deeper inspection, the MongoDB database (exposed on port 27017) can be queried directly using MongoDB Compass or pymongo.

Example queries from `mongosh`:

```
use dyme
// All pending mutants in project 3
db.mutants.find({ status: "pending", id_project: 3 })

// All scavenged data for mutantID 55 of project 3
db.processed_data.find({ id_project: 3, mutantID: 55 })
```

3.9 Exploring Results

Once mutants reach `complete` status, results are accessible through the **Toolbox for Comparative Analysis (TCA)**. Navigate to your project and open the analysis view.

The TCA provides superposed comparative analysis of mutants selected for exploration:

- **Binding free energies (MMPBSA/MMGBSA):** tabular and graphical ΔG comparisons across all mutants, sortable and filterable by sequence or energy range.
- **Per-residue and pairwise decomposition:** heatmaps showing the energetic contribution of individual residues to binding, enabling identification of changes in interaction.
- **Intermolecular contacts:** contact frequency maps derived from MD trajectories, highlighting stable and transient interactions.
- **Water sites:** positions where water molecules mediate protein–protein or protein–DNA contacts, averaged over the simulation ensemble.
- **RMSD profiles:** structural stability of each mutant over the course of the simulation.
- **Best-pose PDB files:** superposed structures of the lowest-energy conformation from each simulation (`output_bestpdb.pdb`), ready for structural inspection in PyMOL, UCSF ChimeraX, or similar tools.

All raw trajectory files (HDF5 format), MMPBSA output files, and topology/coordinate files are stored on the shared filesystem under:

```
/dyme_root/projects/<project_id>/mutants/<mutant_id>/outputs/
```

These can be accessed directly for custom downstream analysis using MDAnalysis, or any compatible MD analysis toolkit.

For additional information on how to perform comparative analysis using the DyME TCA, see supplementary material S3 of the DyME publication (Case Study).